

Project Title

SummarIQ – AI-Powered PDF Summarizer Application for Scientific Papers

Project Overview

Objective

The primary goal of this project is to develop a simplified AI system that summarizes research papers in PDF format. The system will extract main points, including technical content like equations, tables, and key findings. For the first semester, the focus will be on text extraction and generating concise summaries using pretrained NLP models.

Scope

In Scope (Semester 1):

- Upload a PDF research paper and extract text using Python libraries.
- Identify and retain key technical content, including equations.
- Generate extractive or abstractive summaries using pretrained NLP models.
- Display the summarized text in a clean interface.
- iOS demo app: Simple interface to upload PDF, process, and show summary .

Out of Scope (Semester 1):

- Handling images, graphs, or charts in summaries.
- Deep understanding of equations (semantic evaluation).
- Multi-document summarization or literature survey generation.
- Real-time summarization or cloud deployment.

Limitations:

- Summaries may omit fine details from complex equations.
- Works best on text-heavy PDFs.
- Dependent on quality of PDF formatting.

AI Techniques and Tools

Core AI Methods (Semester 1):

- **Text Extraction:** *PyPDF2*, *pdfplumber* for PDF → text conversion.
- **Preprocessing:** Remove headers, footers, page numbers; normalize text.
- **Summarization Models:**
 - Pretrained Transformers: *T5-small*, *BART* for abstractive summaries.
 - Extractive summarization: *LexRank* or *sumy* for selecting key sentences.

Programming Libraries:

- `transformers` → NLP models
- `PyPDF2`, `pdfplumber` → PDF text extraction
- `NumPy`, `Pandas` → data processing
- `SwiftUI` → iOS interface
- `Flask` → optional backend API connecting Python model to iOS app

Semester Milestone:

- Extract text from research paper PDFs.
- Identify sections and equations (as LaTeX strings or placeholders).
- Generate concise summaries (3–5 sentences).
- Display summary in an iOS app interface (upload → processing → result).
- Stretch goal: Highlight equations or key points in the summary.

Future Scope:

- **Equation Awareness:** Integrate tools to parse and semantically understand equations.
- **Multi-Document Summarization:** Summarize multiple papers into one literature review.
- **Graph & Table Summarization:** Include tables, charts, and images in summaries.
- **Deep Learning Enhancements:** Fine-tune transformer models on research paper datasets.
- **iOS Deployment:** Convert model to Core ML for offline summarization on iPhone.

Data Plan:

- **Custom Data:** Select 10–20 PDFs from arXiv or other open-access research papers for testing/demo.
- **Public Datasets:**
 - **ScisummNet** – NLP summaries of scientific papers.
 - **SurveySum** – Summaries of multiple scientific papers.
 - **MS2** – Large-scale dataset with documents and summaries.
 - **cPAPERS** – Papers containing equations and tables for summarization.

Stakeholders:

Project Team:

- Satyabrata Das – AI Developer & iOS Developer → Implement AI pipeline, app interface, and integration.

End Users:

- Researchers & students → Quickly read research papers.

- Educators → Summarize content for lectures.

Other Stakeholders:

- Universities → Can use lightweight summarization tools.
- Open-access repositories → Provide research PDFs.

Self-Rating of Confidence (Beginner Stage):

- Transformers (Hugging Face): 3/10 → New to NLP fine-tuning.
- PDF Processing (PyPDF2, pdfplumber): 7/10 → Comfortable extracting text.
- iOS/SwiftUI: 6/10 → Able to create simple app interfaces and integrate backend.

Computing Infrastructure:

Objective & Tasks (breakdown)

- Main purpose: Extract text from research-paper PDFs, preserve technical elements (equations as LaTeX/placeholders), and produce concise 3–5 sentence summaries (abstractive or extractive).
- Tasks:
 - PDF text extraction & cleanup (headers/footers/pagination removal).
 - Detection/marketing of equations and inline math (stored as LaTeX or [Equation] tokens).
 - Summarization (abstract using T5-small / BART-base; extractive fallback using LexRank).
 - iOS demo app: upload → process → show summary & highlighted equation tokens.

Data types

- Primary: plain text extracted from PDFs (including inline math/LaTeX snippets).
- Secondary: metadata (title, authors, abstract), small extracted tables (as text).
- *Out of scope (semester 1)*: image/figures, charts, semantic equation solving.

Performance benchmarks (targets / decisions)

- Latency (inference for a single PDF summary): **≤ 2 sec** for small models (T5-small) on a GPU-backed inference; **≤ 10 sec** if running on CPU (developer/demo acceptable).
 - *Decision rationale*: tight enough for an interactive demo but realistic for small transformer models.
- Throughput (server, optimistic demo load): **5–20 requests/minute** per single GPU instance for simultaneous users; scale horizontally for more users.
- Accuracy / quality: use ROUGE family (ROUGE-1/2/L) and human evaluation. Targets for semester-1 demo: **ROUGE-1 ≈ 25–40** on scientific-paper summarization tasks (realistic for small models fine-tuned on limited data); supplement with human pass/fail checks vs. paper abstract.

- *Evidence / benchmarks*: ROUGE is the standard automatic metric for summarization. See the original ROUGE paper for metric definitions. Summarization research shows the BART/T5 family perform differently by size and dataset; BART / BART-large variants report strong ROUGE on CNN/DailyMail and scientific datasets (see HuggingFace BART model page and evaluation studies).

Deployment constraints

- Primary deployment for semester-1: **cloud-hosted inference** (Flask API) with iOS front-end calling it. Optionally run inference locally for development.
- Limits: student/demo budget (keep cloud GPU time minimal), mobile app should not require heavy on-device models in Sem-1 (Core ML conversion is a future stretch goal). Network: assume typical Wi-Fi / mobile latency; memory: aim to keep model memory <12 GB for easy cloud GPU choices.

Options considered

- Model choices: T5-small (60M) for lightweight, fast inference vs BART-base/large for higher quality but heavier resource needs.
- Deployment targets: cloud GPU inference (recommended) vs CPU-only (cheaper but slower) vs on-device Core ML (future).

Tradeoff statement

- Prioritize **developer speed & cost** for Semester 1 — choose **T5-small or BART-base** fine-tuned on a few dozen papers for a usable demo (speed/cost) rather than maximizing absolute summarization quality (which would require larger models & more compute).

Risk / trigger

- If observed latency for summaries (on chosen cloud instance) > **2s median** or > **5s 95th percentile**, or if ROUGE/human eval is substantially lower than expected, switch to: (a) stronger GPU/cloud instance for inference (if budget allows), or (b) optimize model (quantize/distill) to meet latency targets.

Evidence / references

- ROUGE evaluation: Chin-Yew Lin (2004). [ACL Anthology](#)
- Model performance and comparisons for BART/T5: HuggingFace BART eval page and comparative papers. [Hugging Face+1](#)
- MLPerf inference helps guide expectations for inference throughput/latency at datacenter scale. [MLCommons+1](#)

Hardware requirements planning:

Decision :

- Use a single **cloud GPU** node for fine-tuning (examples: NVIDIA A10 / A100 / equivalent) or an on-prem GPU like **NVIDIA RTX 4090 / RTX 6000 / A5000** if you have access. For T5-small or BART-base fine-tuning on tens of papers, a single 24–48 GB GPU is sufficient; for larger experiments use A100-class hardware.
- For initial experiments (T5-small): a single GPU with **$\geq 12\text{--}16$ GB VRAM** is sufficient. For BART-large or larger models choose **≥ 40 GB VRAM**.

Alternatives considered:

1. Local workstation with RTX 3090 / 4090 (cheap, one-time purchase) — good for small runs.
2. Cloud GPU instances (AWS/GCP/Azure with A10/A100) — flexible, pay-as-you-go, good quota management.

Evidence: T5-small $\approx$ 60M params (lightweight) and BART-large $\approx$ 400M params — parameter counts inform VRAM needs. (T5-small = 60M). [Hugging Face+1](#)
MLPerf gives datacenter-level inference throughput comparisons (useful when choosing GPU families for production). [MLCommons](#)

Minimum specifications (estimates):

- CPU (training node): 8 vCPUs (dev), 16+ vCPUs for faster data pipeline.
- RAM: 32–64 GB for training/inference servers.
- GPU VRAM:
 - T5-small / BART-base fine-tuning: **12–24 GB** VRAM adequate (fp16/bf16).
 - BART-large / heavier fine-tuning: **40+ GB** VRAM (A100 / A40 / similar).
- Disk: 200 GB SSD for datasets, models, caching; 1 TB if you keep many checkpoints.

Inference hardware (production/demo):

- For interactive demo: single GPU instance (e.g., A10 or T4-level) for low cost; for heavier load use multi-GPU inference with autoscaling. CPU-only fallback possible but expect latency to increase $\sim 3\text{--}10\times$ vs GPU.

Deployment path:

- Semester 1: Cloud inference (single GPU instance) + Flask API.
- Later: either convert to Core ML for on-device inference (small models only) or deploy to multi-GPU cluster for production.

Risk / trigger:

- If cloud GPU costs exceed budget by 25% $\rightarrow$ move training to preemptible/spot instances or downscale model (T5-small vs T5-base), or use mixed precision & gradient accumulation.

Software environment planning:

Decision:

- **OS:** macOS (standard for ML tooling)
- **Frameworks:** PyTorch + Hugging Face (transformers, datasets, accelerate, peft)
- **PDF Processing:** pdfplumber / PyPDF2
- **NLP Preprocessing:** regex, NLTK, spaCy
- **Summarization Models:**
 - Abstractive → T5-small, BART
 - Extractive → LexRank / sumy
- **Backend & Serving:** FastAPI / Flask + gunicorn + uvicorn; Docker for reproducibility (Kubernetes optional later)
- **iOS App:** SwiftUI for frontend; optional Python API backend

Options considered:

- Frameworks: PyTorch vs TensorFlow. (PyTorch + HuggingFace chosen for developer ergonomics & community examples for T5/BART.)
- Containers: Docker for dev vs direct VM. Docker chosen for reproducibility.

Evidence / references:

- HuggingFace documentation on T5/BART model use and pretrained checkpoints.
[Hugging Face+1](#)

Tradeoff statement:

- PyTorch + HF = faster prototyping and many community examples (gain productivity) but might require extra work to integrate certain managed cloud services that favor TensorFlow in some docs (a minor friction).

Risk / trigger:

- If model-serving latency is poor in Dockerized Flask, consider switching to specialized serving stacks (TorchServe, NVIDIA Triton) or using managed inference services.

Cloud resources planning:

Decision: provider & services

- **Primary (recommended for students/demo):** AWS or GCP (choose based on student credits / familiarity). Use **GPU-enabled VM** or managed ML service: AWS SageMaker (for easy training + deployment) or GCP Vertex AI. For a low-cost demo you can use a single GPU VM (spot/preemptible) for training and a small GPU instance for inference.
- Example service map:
 - Training: spot/preemptible GPU instances for cost savings.

- Inference: small GPU VM with autoscaling or managed endpoint (SageMaker endpoint / Vertex AI endpoint).
- Storage: S3 / Google Cloud Storage for PDFs and model artifacts.
- Database: small managed Postgres or Firebase for user metadata.

Options considered:

- AWS SageMaker vs GCP Vertex AI vs Azure ML. (All support Hugging Face workflows; choose a provider based on credits / quotas.)
- Serverless inference (cheaper for low traffic) vs containerized VMs (more control).

Cost estimation:

- Use provider calculators (estimate per-hour GPU costs). Example: training on A100 costs much more than on T4/A10; for a semester project, prioritize T4/A10 or spot instances for training. (Exact numbers depend on the provider/region — run cloud pricing calculators when finalizing.)

Evidence:

- Cloud ML managed offerings and pricing calculators exist; MLPerf / vendor docs can help choose GPU families for throughput. [MLCommons+1](#)

Tradeoff statement:

- Choose **flexibility & lower upfront cost** (cloud VMs + spot instances) over owning hardware. This avoids high capital expense but adds ongoing operational cost.

Risk / trigger:

- If monthly cloud spend > budget threshold (e.g., 25% over planned), move training to preemptible instances, reduce training time (fewer epochs / smaller models), or use local GPU if available.

Scalability & performance planning:

Scaling strategy:

- **Horizontal scaling on inference:** scale out multiple inference replicas behind a load balancer (Kubernetes or managed endpoints). For higher throughput, use model sharding / multi-GPU inference.
- **Auto-scaling policy:** scale replicas based on request latency or CPU/GPU utilization. For demo, manual scale-up is acceptable.

Optimization techniques:

- **Mixed precision / fp16 or bf16** during training & inference to cut memory and speed up inference.
- **Quantization (8-bit or 4-bit)** for inference to reduce memory & speed up (use bitsandbytes or ONNX runtime with INT8).

- **Knowledge distillation:** distill a larger teacher model into a smaller student model for faster inference with similar quality.
- **Pruning** for further size reduction if necessary.
- **Caching:** cache summaries of identical PDFs (or repeated requests) to avoid re-running inference.
- **Batching:** handle small batches of simultaneous requests to improve GPU utilization.

Performance monitoring & tooling:

- Track latency (p50/p95), throughput (requests/sec), error rate, GPU/CPU utilization, and memory consumption.
- Tools: Prometheus + Grafana for metrics, or managed observability (CloudWatch, Stackdriver). Use APM tools for tracing (OpenTelemetry).
- Regular profiling with NVIDIA Nsight or PyTorch profiler during optimization passes.

Options considered:

- Vertical scaling (bigger GPU) vs horizontal scaling (more replicas) — combine: use bigger GPU for low-latency single-large-instance needs; horizontal scaling for many concurrent users.

Tradeoff statement:

- Prioritize **low-latency interactive experience** for early demos (use single GPU with fp16 + batching) rather than maximal throughput. Later, scale horizontally and add quantized models for cost efficiency.

Risk / trigger:

- If p95 latency > target (e.g., > 5s for interactive demo) → apply quantization+distillation or increase GPU capacity. If cost per inference > target, switch to distilled/quantized student models or scheduled batch processing.

Security, Privacy, and Ethics:

In this project, trustworthiness is considered at every stage of the AI lifecycle — from defining the problem to maintaining the deployed system. The goal is to make sure the summarization system is fair, secure, transparent, and useful for all users.

Problem Definition:

- **Strategy:** Identify ethical risks (e.g., biased summaries, misrepresentation of scientific content).
- **Plan:** Reflect on potential harms (like oversimplifying technical papers or removing critical details).
- **Tool/Method:** Use the **Data Ethics Canvas** to map possible risks and stakeholder concerns before implementation.

Data Collection:

- **Strategy:** Ensure data used for training and testing is representative and privacy-conscious.
- **Plan:** Collect scientific papers (e.g., arXiv subset), anonymize sensitive metadata, and balance datasets.
- **Technical Implementation:** Use **Snorkel** to generate synthetic examples that reduce underrepresentation of certain categories of papers.

Model Development:

- **Strategy:** Build fair and explainable models, and test robustness.
- **Plan:** Use **T5-small** and **BART** for abstractive summarization, while monitoring for bias in how technical content is summarized.
- **Technical Implementation:** Apply **Fairlearn** to compare model outputs across subsets of data (e.g., physics vs. medical papers) and check fairness in summaries.

Deployment:

- **Strategy:** Deploy the summarization service securely and allow user feedback.
- **Plan:** Containerize the model with Docker, expose it through a **FastAPI** endpoint, and restrict API access with authentication.
- **Tool:** Use **BentoML** to manage serving and monitoring of the summarization model.

Monitoring and Maintenance:

- **Strategy:** Continuously check performance, detect drift, and gather user feedback.
- **Plan:** Track summary quality over time and retrain models if performance drops.
- **Tool:** Use **NannyML** for drift detection, and run small user studies (students rating helpfulness of summaries) to guide improvements.

Human-Computer Interaction (HCI):

For this project, HCI requirements are defined early to ensure that the summarization system is simple, useful, and accessible for students, researchers, and educators.

Understand User Requirements:

- **Approach:** Conduct small surveys with students and faculty (Google Forms) to learn how they currently read and summarize papers.
- **Insights Expected:** Users want faster ways to skim papers, extract main points, and identify key equations.
- **Strategy:** Semi-structured interviews with 3–5 students and 1–2 faculty to capture both detailed feedback and common pain points.
- **Example Tool:** Affinity diagramming in Miro to group feedback (e.g., "time-saving," "clarity of equations").

Create Personas and Scenarios:

- **Personas:**
 - *Student Persona:* Undergraduate student who needs quick summaries for class assignments.
 - *Researcher Persona:* Graduate student or PhD scholar who wants technical summaries with equations retained.
 - *Educator Persona:* Lecturer preparing lecture notes and needing quick overviews of papers.
- **Scenarios:**
 - A student uploads a PDF → system generates a 3–5 sentence summary.
 - A researcher tests if equations are correctly preserved in summaries.

Conduct Task Analysis:

- **Main Task:** Upload a PDF and receive a summary.
- **Subtasks:**
 - Select and upload PDF (via file picker in iOS app).
 - Wait during the "Processing" phase (loading feedback shown).
 - Read generated summary.
 - *Optional:* Copy/paste summary or share it.
- **Method:** Use Hierarchical Task Analysis (HTA) to break tasks into smaller actions (e.g., tapping upload → file picker → selecting file).

Identify Accessibility Requirements:

- **Plan:** Ensure the app is usable for people with diverse needs.
- **Features to include:**
 - Large, clear buttons and readable fonts.
 - Screen reader support for reading summaries.
 - Keyboard navigation support (future web version).
- **Tool:** Use WAVE or Axe to check accessibility standards (WCAG compliance) if building a web version later.

Outline Usability Goals:

- **Goals for Semester 1:**
 - Reduce time to understand a paper by 50% compared to manual reading.
 - Summaries should be readable in under 1 minute.
 - Provide at least 70% user satisfaction in small user testing (feedback survey).
- **Measurement:** Short feedback survey after demo, tracking metrics like task completion time and satisfaction.

Git Repo link:

<https://github.com/Satyabratadas/SummarIQ.git>

User Interface(IOS):



